

StayOS - Comprehensive System & Workflow Documentation

1. Executive Summary & Objective

StayOS is a robust, multi-tenant Hotel Management Software-as-a-Service (SaaS) platform built to handle the rigorous demands of modern hospitality businesses. It provides an all-encompassing suite for administration, front desk, human resources, analytics, and housekeeping.

2. Platform Administrator Module (Super Admin)

The Super Admin dashboard is the central command center for the SaaS owner to monitor and manage all subscribed hotels.

2.1 Admin Dashboard (Platform Overview)

- **Purpose:** High-level bird's-eye view of the platform.
- **Workflow:** The admin logs in and immediately sees active hotels, Monthly Recurring Revenue (MRR), total active trials, and expired trials. Trend lines show growth in hotel onboarding.

2.2 Hotels Management Tab

- **Purpose:** Onboard and manage hotel tenants.
- **Workflow:**
 1. Click "Add Hotel".
 2. Input hotel details, address, and the root manager's email/password.
 3. Assign a subscription tier (Starter, Pro, Enterprise).
 4. The system automatically creates an isolated database environment (logically) via Mongoose schemas for this hotel. The hotel manager can now log in.

2.3 Subscriptions & Billing Tab

- **Purpose:** Oversee SaaS payments and trial states.
- **Workflow:** The admin tracks which hotels are currently in their 24-hour/14-day free trial. If a trial expires, the hotel is automatically locked out. The admin can manually approve a payment, extend a trial, or view webhook receipts from the Razorpay integration.

2.4 Multi-Branch Tab

- **Purpose:** Grouping hotels under corporate umbrellas.
- **Workflow:** Admins can link 'Hotel A' and 'Hotel B' to a single enterprise entity, allowing the enterprise owner to view consolidated revenue reports across branches.

2.5 Admin Settings & Audit Logs

- **Purpose:** Security and platform configuration.
 - **Workflow:** Adjust global notification templates, manage sub-admin roles (e.g., Support Staff vs Super Admin), and review audit logs for compliance to see who modified subscription statuses.
-

3. Hotel Operations Module (Property-Level)

When a hotel manager or receptionist logs in, the RBAC (Role-Based Access Control) system determines which of the following tabs they can access.

3.1 Hotel Dashboard

- **Purpose:** Daily operational snapshot.
- **Workflow:**
 1. The Receptionist views today's Check-ins and Check-outs in a quick table.
 2. The live Occupancy Meter shows the percentage of occupied vs available rooms.
 3. Pending Payment Alerts flag guests who owe money.
 4. Housekeeping Grid allows quick toggling of a room from "Dirty" to "Clean".

3.2 Smart Check-In / Check-Out Tab

- **Purpose:** Streamlining the guest arrival and departure process.
- **Workflow (Check-In):**
 1. Enter Guest Details (Name, Phone, Email, ID Proof upload via Multer).
 2. Select Dates and assign an Available Room.
 3. Enter the agreed Tariff and any Advance Payment collected.
 4. System marks the room as "Occupied" and creates an active Booking record.
- **Workflow (Check-Out):**
 1. System aggregates the Room Tariff, Restaurant Bills, Laundry, and Travel charges into a Master Folio.
 2. Staff records the final payment.
 3. System marks the booking as "Checked-Out" and changes the Room status to "Dirty" for Housekeeping. A real-time socket event notifies Housekeeping.

3.3 Rooms Management Tab

- **Purpose:** Real-time inventory control.
- **Workflow:** Visual grid of all physical rooms. Staff can click a room to place it "Under Maintenance", effectively removing it from the available inventory for new check-ins until maintenance resolves the issue.

3.4 Bookings Tab

- **Purpose:** Ledger of all historical and future reservations.

- **Workflow:** Staff can search past guests, modify future check-in dates, process cancellations, or issue refunds.

3.5 Billing & Revenue Reports Tab

- **Purpose:** Financial tracking and End-of-Day (EOD) reconciliation.
- **Workflow:** Managers review the Daily Revenue Report generated by the system. It breaks down revenue by source (Rooms, POS, Laundry). Managers can export these reports to Excel (.xlsx) for accounting software.

4. Guest Services & POS Integrations

4.1 Guest CRM & Loyalty

- **Purpose:** Enhancing guest retention.
- **Workflow:** The system automatically builds a profile for every guest. It tracks total nights stayed and total revenue generated. Managers can use this to apply loyalty discounts for returning VIP guests.

4.2 Restaurant POS, Laundry & Travel Desk Tabs

- **Purpose:** Ancillary revenue management.
- **Workflow:**
 1. A guest orders room service.
 2. Staff opens the Restaurant POS tab, selects the items, and clicks "Bill to Room".
 3. The system utilizes the `ManualItems` API to seamlessly attach this charge to the guest's active booking folio. No cash needs to change hands until checkout.

5. Staff Management & HR

5.1 Employees & Attendance Tabs

- **Purpose:** Shift tracking and HR.
- **Workflow:** Managers add staff profiles and assign roles (e.g., Reception, Housekeeping). Staff use the system to clock in/out, feeding data into the Attendance tab.

5.2 Housekeeping & Maintenance Tabs

- **Purpose:** Maintaining property standards.
- **Workflow (Housekeeping):** Housekeepers see a tablet-optimized view of only "Dirty" rooms. Upon cleaning, they tap "Clean". The Dashboard instantly reflects this, and the room becomes available for the front desk to assign.
- **Workflow (Maintenance):** If a TV is broken, staff logs an issue. Maintenance sees the ticket, repairs the TV, and marks the ticket as "Resolved", returning the room to the active pool.

5.3 Payroll Tab

- **Purpose:** Salary distribution.
- **Workflow:** At month-end, the system calculates payroll based on base salary minus unpaid absences derived from the Attendance tab. Managers approve and generate paylips.

6. System Architecture & Technologies

- **Frontend:** React (Vite) Single Page Application. Context API manages the global state (user sessions, WebSocket connections).
- **Backend:** Node.js & Express.js handling REST API requests.
- **Database:** MongoDB (via Mongoose) ensuring strict multi-tenant data isolation using `hotelId` indexing.
- **Real-time Engine:** Socket.io guarantees that when Reception checks out a guest, Housekeeping immediately sees the room turn "Dirty" without refreshing the page.
- **Security:** JWT authentication, bcrypt password hashing, express-mongo-sanitize, and Helmet HTTP headers.